Lesson 4 Teacher's Checklist / Prep Guide

Complete before class

VIDEO GAME CREATION

Self-Contained Lesson Assets

Per Lesson Production Standards Rule 3, nothing this lesson needs should require a student to search for or download something mid-class. Everything below is already prepared and included in this lesson's Assets folder - no download required.

- ☐ New placeholder enemy sprite already generated (enemy_invader_red.png, 128x128 with transparency, a pixel-art "invader" silhouette in red) - distinct in color and shape from the player and bullet sprites.
- ☐ New sound effect already generated (sfx_hit.wav, a short impact/explosion burst - deliberately distinct from Lesson 3's sfx_shoot.wav so firing and hitting don't sound the same) for the Level 3 "add a sound" idea.
- ☐ Lesson 2 and 3's Assets folders (player sprites, bullet_yellow.png, sfx_blip.wav, sfx_shoot.wav) are still needed too, since this lesson builds on the same ongoing [YourName]Game project rather than starting fresh.
- ☐ Copy this lesson's Assets folder onto the shared classroom drive / USB backups alongside the handouts.

Continuity From Lesson 3

Same pattern as last week - students open their existing project rather than starting fresh.

- ☐ Remind students to open their own [YourName]Game project from Lesson 3, not create a new one - the enemy and collision detection are added on top of the shooting mechanic they already built.
- ☐ If a student's Lesson 3 project is lost or broken, have a pre-built "Lesson 3 complete" starter available per engine (from the Lesson 3 Teacher's Keys) so they aren't blocked.

Engine Verification

This week's core new concept - collision/overlap detection - works differently enough across engines that it's worth a quick personal test run before class, even under the "charge through" pacing this course is using for Lessons 4-15.

- ☐ Confirm the demo laptop's Lesson 2 and 3 exercises still work for all 8 engines - re-run any that haven't been touched in a few weeks.

- ☐ Pre-build the full Scratch collision exercise on top of a finished Lesson 3 project, so you have a working example ready to show.
- ☐ Godot: confirm an Area2D's area_entered signal actually fires between two Area2D nodes (not body_entered, which is for PhysicsBody2D/TileMap nodes instead) in your installed version - this is the one new physics concept this lesson introduces.
- ☐ Unity: confirm a Rigidbody2D (Body Type = Kinematic) plus two Is Trigger colliders actually fires OnTriggerEnter2D in your installed Editor version - test this once before class, since a missing Rigidbody2D on either object is the most common silent failure.
- ☐ GameMaker: quick-test a Collision event between two throwaway objects in a test room before class.
- ☐ GDevelop: confirm the built-in "is in collision with" condition is available between two Sprite objects in your installed version (it should be, by default).
- ☐ TIC-80: there's no built-in collision system here - confirm the rectangle-overlap math in the Key matches your chosen sprite size if you use anything other than the default 8x8 sprites.
- ☐ 001 Game Creator (Windows demo laptop): confirm the "Collided with actor" trigger and the relate keyword are available and behave as documented in the Actor Trigger editor for your installed version - this is a different trigger than Lesson 3 used.
- ☐ If any advanced-track students are on Unreal this week, practice setting Collision Presets to OverlapAllDynamic on a Shape component once so it's a familiar click during class - the default BlockAllDynamic preset silently blocks Event ActorBeginOverlap from ever firing, and it's easy to mistake for a wiring problem.

Materials

- ☐ Print or queue digital copies of all 8 Student Handouts (or confirm students can access their engine's version electronically).
- ☐ Have the 8 Teacher's Keys on hand (instructor reference only - do not distribute) in case a student needs a worked example beyond Level 1.
- ☐ Bring: demo laptop, projector cable/adapter, handout copies, Assets folder (on USB as backup).

Known Gotchas This Week

- Unity: trigger events need a Rigidbody2D on at least one of the two colliding objects - this is the single most common "nothing happens" bug this week, since it's easy to add colliders and forget the Rigidbody2D entirely.
- Godot: Area2D only detects other Area2D nodes via area_entered (or PhysicsBody2D nodes via body_entered) - a plain Sprite2D with no Area2D/CollisionShape2D child is invisible to collision detection entirely, which is why both bullet.tscn and enemy.tscn need their own Hitbox this week. Separately, this is the first week students position something themselves in the 2D editor (dragging Enemy into main.tscn) - Godot's origin is the top-left corner with Y increasing downward, so a student used to a different convention can place Enemy well off-screen with no error at all. It'll still show up in the Remote tab while running (proving it exists), just not on screen - point them at Transform > Position in the Inspector, not the Texture, first.
- Unreal: a fresh Shape's default Collision Preset (BlockAllDynamic) makes it physically block other dynamic objects instead of generating an overlap event - if Event ActorBeginOverlap never fires, check Collision Presets on BOTH BP_Enemy and

BP_Bullet before suspecting the node wiring. A second, separate snag: BP_Bullet's Add Actor World Offset movement node from Lesson 3 needs its Sweep box checked, or a fast-moving bullet can pass straight through the enemy each frame without the move ever testing for what it passed into.

- 001 Game Creator: non-Solid Actors generate no collisions at all - if an enemy or bullet was created with "Solid" unchecked (or inherited that setting from its Template), nothing will ever detect it.
- TIC-80: there's no built-in collision system - the rectangle-overlap check in the Key is deliberately simple (axis-aligned bounding box), which is normal for this engine and not a shortcut being taken.